qcode

User Manual

A terminal-first AI coding assistant.
This guide shows how to use qcode day to day.

Version dev - October 14, 2026
Usage guide for people using qcode.
No setup internals or code architecture inside.

In this manual: install once, tour the screen, try practical use cases, clarifying questions, switch models, use agents, plan, skills, memory, web tools, sessions, browser control, and settings.

How to use this manual

Read chapters 1-3 to start. Keep chapters 4 and 15 nearby as a command reference. The Use cases pages after chapter 3 offer complete example workflows. Chapters 5-12 are task guides you can read when you need them: models, agents, planning, skills, memory, web, sessions, and phone/browser control.

- Words in `code style` are things you type, for example `/model` or `qcode --demo`.
- Figures are illustrative example screens. Your colors and sizes may differ slightly.
- The prompt line shows `>` when idle, `(Plan)>` in Plan mode, `(Skill plan)>` in Skill Plan mode, and `(Steer)>` while busy.
- Esc goes back one level. Ctrl+C closes a command view; agents continue working. In the composer, Ctrl+C cancels active work.

Tip

Run qcode --demo for the interactive tour with automatic queued prompts. Add a quoted prompt for a one-shot text demo. Both use mocked tools and leave your workspace untouched.

1. Getting started

Install and first run

qcode is one program you run in your terminal. Build it once, then start it in any project folder.

```
# Build from source (needs Go 1.22+)
make build
./bin/qcode
```

The remaining shell examples assume qcode is on your PATH. From this source checkout, use ./bin/qcode instead. make build creates the binary but does not install it.

Pick the assistant you want to use. The two common choices are a local model (Ollama) or a hosted model (OpenAI or OpenCode Go).

```
# Local model (default provider)
ollama pull qwen2.5-coder:7b
qcode --model qwen2.5-coder:7b

# Hosted model
export OPENAI_API_KEY=...
qcode --provider openai --model gpt-5

# No setup interactive tour
qcode --demo
```

Tip

If you only want to learn the keys and screens, start with --demo. It runs a scripted tour with fake tools and queued prompts.

Update

Use the shell command below to install the latest release. It is a qcode CLI command, not a slash command.

```
qcode update
qcode update --arch arm64
```

2. Touring the terminal

qcode fills the terminal but keeps your normal scrollback. Tabs sit at the top, with the latest received prompt highlighted directly beneath them. The transcript scrolls in the middle; pending work, the editable composer, task indicator, and status bar stay at the bottom.

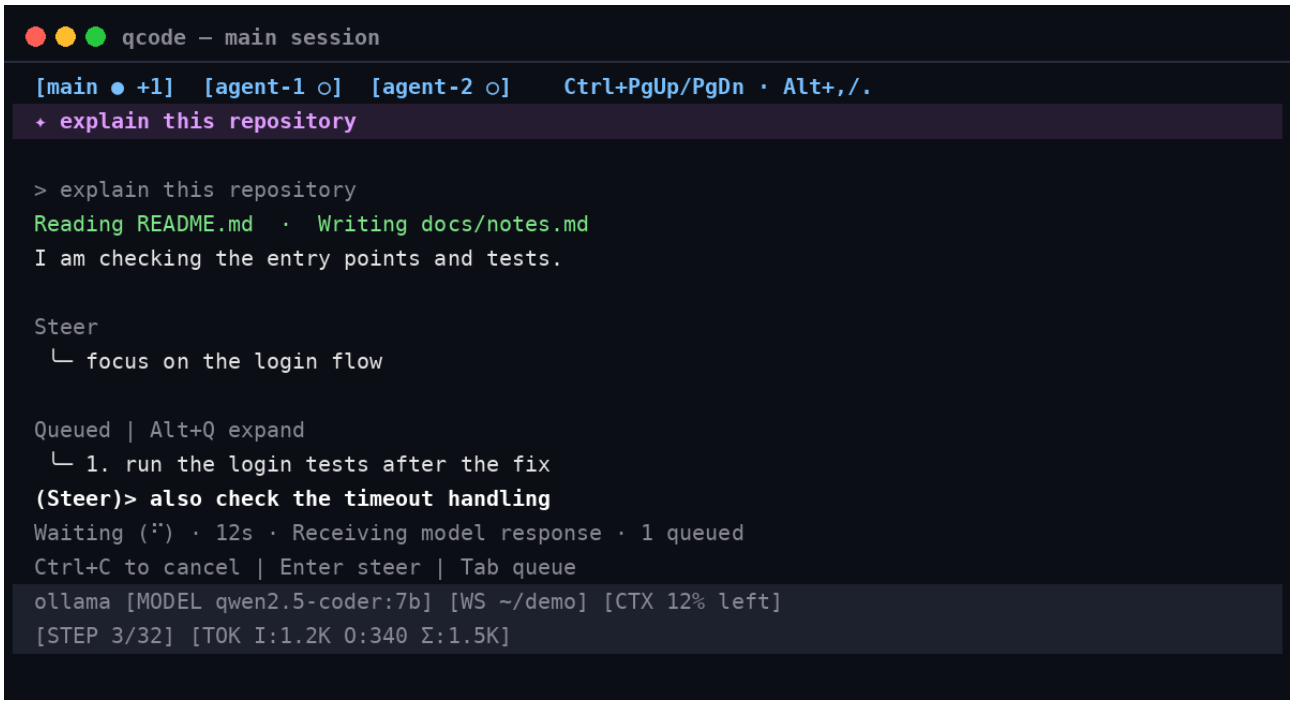


Figure 1: Main screen. The highlighted prompt stays below the tabs; pending steering and queued tasks appear above the composer.

What you see

- Tabs: `main` is always there. Extra agents appear as new tabs (up to 20 total).
- Latest prompt: a star marks up to three highlighted lines beneath the tabs. It changes when a task starts or steering is delivered. Pending input does not change it; /clear and completion keep it, /new clears it, and /resume restores it. Small windows show fewer lines.
- Transcript: streamed answers with Markdown, tables, and tool activity like `Reading ...` or `Writing ...`.
- Task indicator: Waiting for model response, Receiving model thinking, or Receiving model response, with elapsed time for that operation. The timer resets when the phase changes. Tools appear in the transcript; questions show Waiting for input.
- Pending area: shows the current steer separately from FIFO tasks.
- Prompt: `>` idle, `(Plan)>` planning, `(Skill plan)>` skill design, `(Steer)>` working. You can keep typing while work runs.
- Status bar: provider, MODEL, WS folder, CTX percent left, STEP n/max, and TOK input/output plus total when it fits. THINK appears when set, MODE for planning or interactive questions, and REMOTE while browser control runs. Hide parts with /statusline.

Keys you will use daily

- Type `/` to see up to five matching commands. Type more to filter, Tab to complete the first match, Esc to close.
- Left/Right edits input. At the bottom of output, Home/End move to input ends. Ctrl+Left/Right jumps by word; Ctrl+W deletes a word; Ctrl+A/E moves to input ends.
- PageUp/PageDown scrolls history with two overlapping rows of context, even while the agent works. Scrolling pauses the live view; PageDown to the bottom resumes it.
- When the view is above the bottom, Home/End jump to oldest/latest output. At the bottom they edit the prompt again. Browser text fields keep editing keys.
- Alt+Q opens pending work. Up/Down selects, PgUp/PgDn scrolls, Delete removes, and Esc closes.
- Ctrl+C in the composer cancels the running prompt; the next queued prompt can start. In a command view, it closes the view while agents keep working.
- NO_COLOR=1 disables colors; QCODE_ASCII=1 uses ASCII glyphs.

3. Asking, queueing, and history

Type a request and press Enter. While one request runs, you can type the next one. Enter steers the current task; Tab queues a separate task. Steering waits for a response or tool, replaces pending steering, and withdraws unanswered interactions.

The pending panel separates Steer from Queued tasks with tree connectors. Use Alt+Q to expand it, Up/Down to select, Delete to remove an undelivered item, and Esc or Alt+Q to return to typing. Once steering starts replanning, it cannot be removed. Completed tool changes remain in place when you steer or cancel.

- One prompt runs at a time per agent. Other agents are not blocked.
- The tab and task line show the queued task count.
- The newest pending steer replaces earlier pending steering. If the task ends before your steer is accepted, qcode rejects it and keeps the draft. Cancellation or failure drops undelivered steers; queued tasks still continue.
- Tab still completes matching slash commands. During manual /compact, Enter cannot steer and keeps your draft; Tab can queue a separate task.
- Use `/history` to find an old prompt. Pick one to re-read its final answer.
- Use `/new` to clear the active tab's conversation, latest prompt, usage, and plans; it restores web tools to on outside the sandbox and off inside it, and removes added directory grants. Models, selected skills, and the questions toggle stay.
- Use `/compact` to summarize old context. Automatic compaction starts at 80% use when capacity is known; change auto_compact_threshold or disable_auto_compact in config.toml.

Tip

Example: submit `fix the login timeout` with Enter. While it runs, submit `preserve the existing API` with Enter to steer it, or `run the login tests after the fix` with Tab to queue a separate task.

When qcode asks you back (/interactive)

Normal work can pause to ask one focused question when an ambiguity would otherwise need several broad searches. Turn this on per agent with `/interactive on`, off with `/interactive off`, or check the setting with `/interactive`. It is off by default. Add `interactive = true` to config.toml to enable it for new terminal sessions.

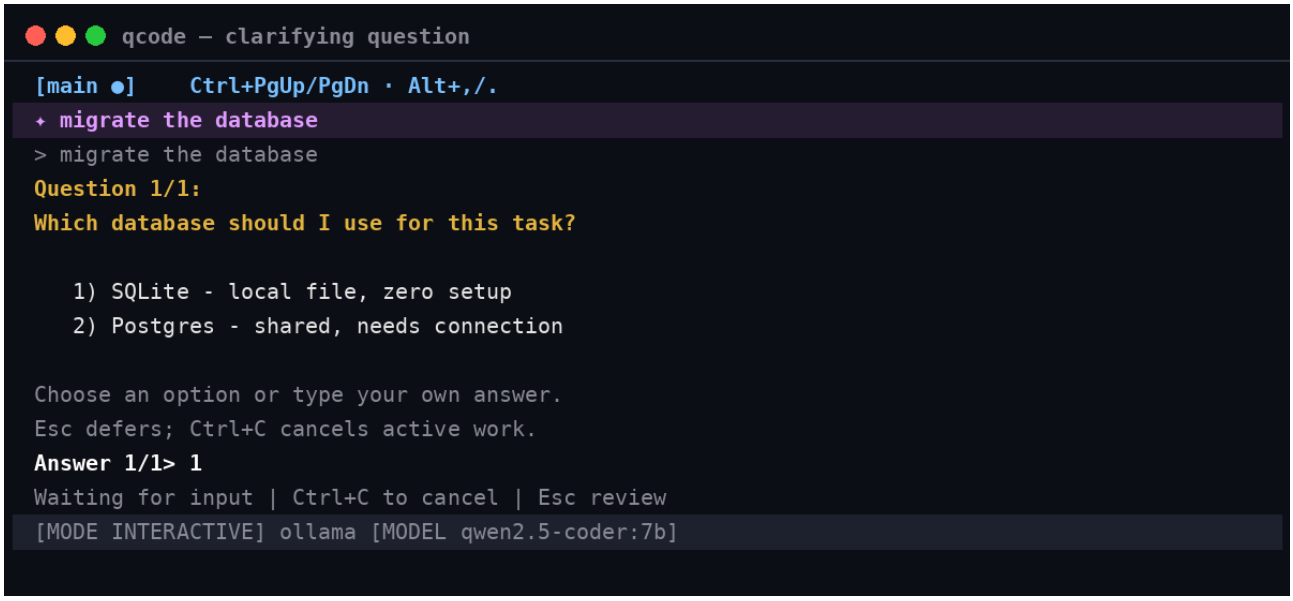


Figure 2: A clarifying question. Type an option number or your own answer. At most 3 questions per prompt.

- One question at a time, with suggested choices or your own custom answer. At most 3 distinct questions per submitted prompt.
- Your answer becomes context for that agent and is included in saved sessions. It is not added to global learning automatically; use /learn to retain a reusable preference.
- The status bar shows [MODE INTERACTIVE] while enabled ([MODE INT] when narrow). Your half-typed draft is saved and restored around the question.
- /interactive on|off saves the default for future terminal sessions from any agent tab. Each existing or resumed tab keeps its own setting; demo changes stay session-only.
- Esc defers a question and opens the composer; Esc returns to it. A steer withdraws unanswered interactions. Ctrl+C cancels active work. The browser can also answer.
- One-shot prompts, piped input, non-terminal runs, and --json-events never wait; qcode just proceeds with available context.

Tip

Turn it on when tasks are ambiguous (which database, which scope). Leave it off for strict hands-off runs.

Use cases: understand and fix

These are example requests, not built-in commands. Replace paths, commands, and symptoms with those from your project. Review the resulting files and command output before accepting a change.

Understand an unfamiliar repository

Goal: find the entry points and learn how to run the project before making changes.

1. Start qcode in the project folder and submit the example below.
2. Ask a follow-up about the part you need to change. Use /history to revisit the explanation.
3. Check the cited files and confirm the suggested commands against the project's README.

```
Explain this repository without changing files. Identify the
entry points, how to run it, and how to run its tests. Cite the
files you used, and list anything you could not confirm.
```

Result to look for: a file-grounded starting guide with clear run and test commands, plus open questions.

Fix a bug and queue the next step

Goal: reproduce a specific failure, make a focused fix, and check the result.

1. Use /interactive on if expected behavior needs clarification; then describe the failure and a reproduction command.
2. While the fix runs, press Tab to queue the follow-up below. It waits in the active agent's FIFO queue.
3. Review the diff with /diff and inspect test output. Ask for an explanation if the checks fail.

```
Fix the login timeout: retry once after a temporary network
failure, but never retry invalid credentials. Reproduce it
with the login tests before changing the handler.

# Press Tab to queue while the first request is running:
Run the login tests after the fix and summarize the results.
```

Result to look for: a small diff and observed test results, with any remaining failure stated explicitly.

Use cases: plan and divide work

Plan a change before editing

Goal: settle scope and review the approach before implementation.

1. On an idle agent, enter /plan and describe the change below.
2. Answer design questions, then use /plan show to review the proposal. Request revisions if needed.
3. Use /plan act only when ready to implement. Review the diff and the planned checks afterward; /plan off leaves without implementing.

```
/plan
Plan pagination for the search results. Inspect the current
API and UI, identify compatibility concerns, and propose
acceptance checks before changing files.
/plan show
/plan act
```

Result to look for: an agreed plan followed by changes that meet its acceptance checks.

Use independent agents for implementation and investigation

Goal: keep a focused implementation moving while another tab investigates a separate question.

1. Give main the implementation task. Use /agent new to pick a model for a helper; rename its displayed ID with /agent rename <id> review-tests.
2. In the helper tab, ask for a read-only review of relevant tests and missing edge cases. Switch tabs with Ctrl+PageUp/PageDown.
3. Bring the helper's findings back to main, request the needed checks, and review the combined result.

```
# Main prompt:
Add the agreed pagination behavior to the search endpoint.

# Helper prompt:
Read the search tests without changing files. Identify
missing pagination edge cases and report the file locations.
```

Tip

Agents share the workspace. Give them distinct responsibilities; avoid asking two tabs to edit the same files at once. Each tab has its own history and queue.

Use cases: reuse and continue

Turn a repeated review into a skill

Goal: make a repeatable checklist available in future work.

1. Start /skillplan with the example below and answer questions about scope, inputs, and validation.
2. Review with /skillplan show; request edits until the draft describes your actual workflow.
3. Use /skillplan create to write the reviewed skill, then /skill to enable it before the next review.

```
/skillplan Create a database migration review checklist.
Check rollback behavior, existing data compatibility,
and validation commands. Report risks with file references.
/skillplan show
/skillplan create
/skill
```

Result to look for: an enabled, reusable skill whose instructions and completion checks you have reviewed.

Resume work and share its outcome

Goal: continue a saved conversation and produce a readable handoff.

1. Return to the same project folder, start qcode, and use /resume to choose the saved session.
2. Use /history to find the last completed answer. Ask for current status and remaining work before continuing; interrupted tools are not rerun automatically.
3. Use /export pretty for a shareable HTML conversation, or /export raw when the handoff needs detailed activity.

```
/resume
Summarize the completed changes and remaining work. Verify
the current files before continuing the unfinished task.
/export pretty
```

Tip

To continue from a phone on a trusted network, use /remote and redeem the QR link. The browser controls the same live session; its /export downloads the HTML. Close the connection when finished.

4. Slash commands at a glance

Type `/help` to list commands or `/help <name>` for one command. The leading `/` is optional in the name. Agent, skill, learning, plan, remote, model, tool, and interactive commands are covered in their chapters below.

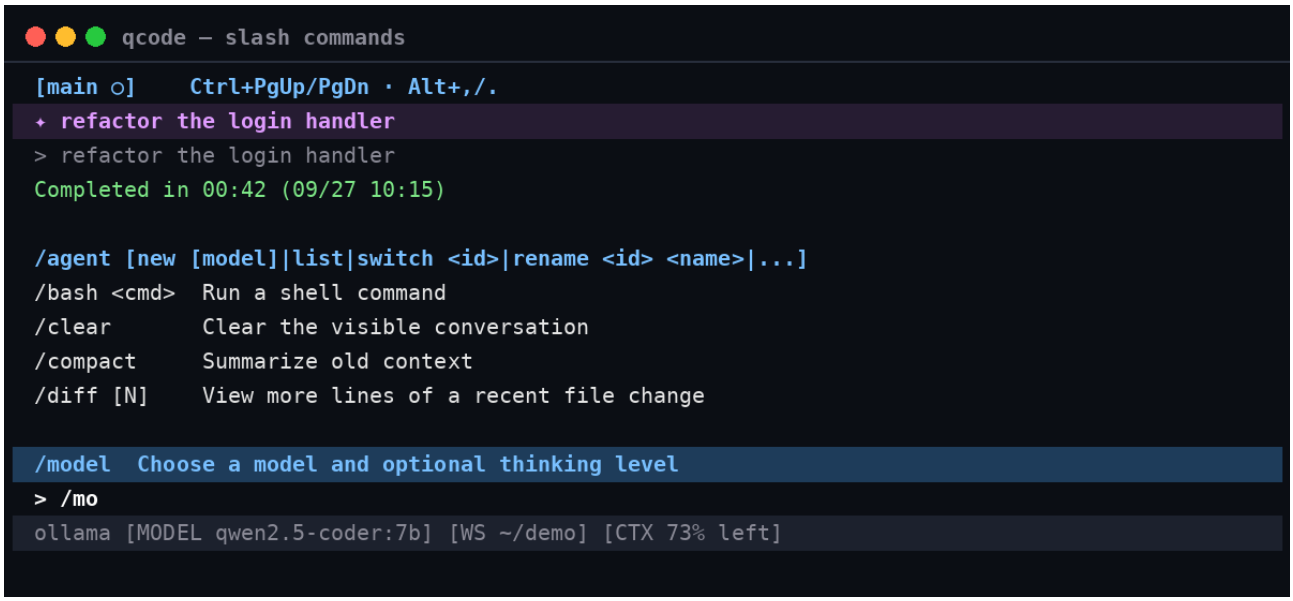


Figure 3: /help lists commands. Typing /mo filters the suggestions to /model; Tab completes the first match.

Command	What it does for you
/help	List commands or explain one command.
/bash <cmd>	Run a shell command in the workspace.
/new	Start a fresh conversation for the active agent.
/resume	Preview, rename, pin, delete, or continue saved sessions.
/clear	Clear the visible conversation and redraw the header.
/history	Browse completed prompts and responses.
/diff [N]	View more lines of saved diff N; omit N for latest.
/verbose	Show or hide detailed action traces.
/maxsteps [N]	Show or change the model-turn limit.
/statusline	Show, hide, or reset status bar parts.
/theme	Preview, apply, and save a terminal color theme.
/compact	Summarize old context to make room.
/export	Export conversations or full transcript as HTML.
/quit, /exit	Save the session and exit qcode.

Diff previews

When qcode writes or edits a file you see a short numbered 10-line preview with added and removed lines. Use `/diff` to expand the latest saved diff or `/diff N` to expand saved diff N, up to the 200-line safety limit. The model still receives plain text.

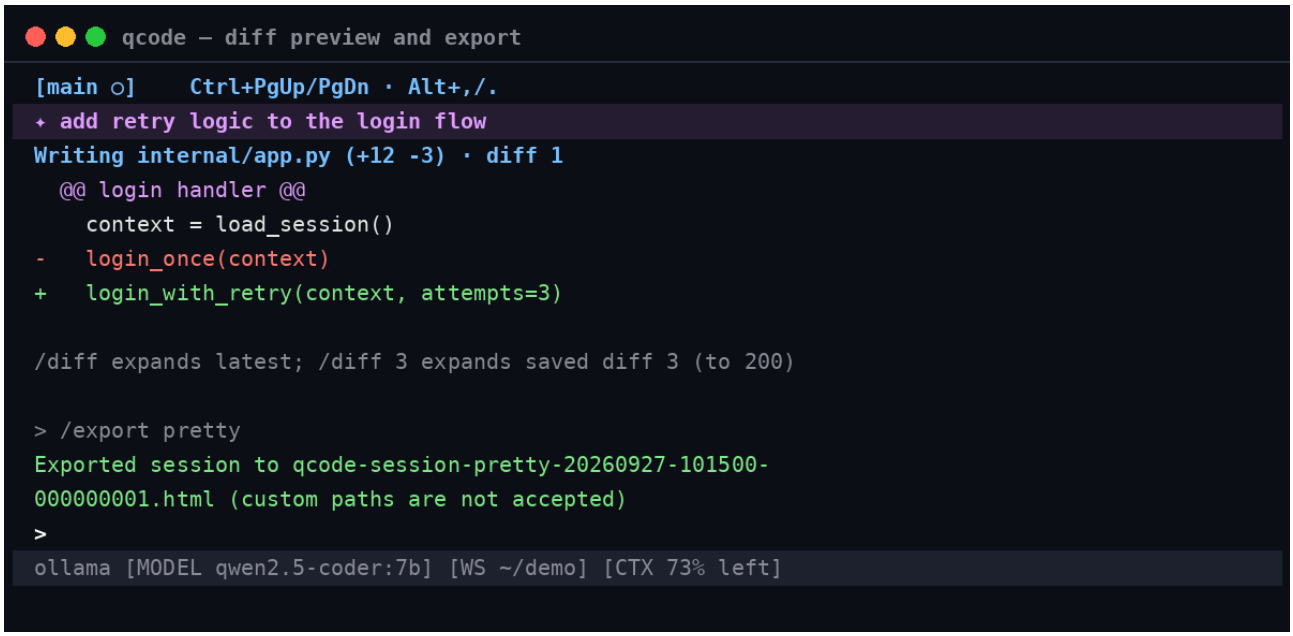


Figure 4: Numbered change preview plus /export pretty saving a readable HTML copy.

Exports

- `/export` or `/export pretty`: one tab per agent, prompt cards that reveal answers. Best for sharing and reading.
- `/export raw`: full styled transcript including tool activity, thinking, errors, and diffs.
- In a pretty export, Show detailed tool events reveals concise activity beside each exchange; it starts unchecked.
- Files are saved as `qcode-session-<mode>-<timestamp>.html` with subsecond precision. Custom output paths are not accepted.
- Terminal exports write in the workspace; browser exports download the same HTML without writing a host file.

5. Choosing models and providers

Use `/model` to switch the assistant for the current tab. The list shows model names from your current provider only, with no provider suffix. Type to search, move with Up/Down and PgUp/PgDn, Enter to pick. You can also run `/model <id> [thinking]` directly. Some models ask a second question for thinking level; Esc returns to the model list and Ctrl+C cancels.



Figure 5: /model picker. Model names only; the list comes from the current provider. An optional thinking level follows (off/low/medium/high/max). Choices depend on the model.

Providers you can use

- Ollama (local default): pull a model first, then `qcode --model qwen2.5-coder:7b`. Thinking uses the model's native think setting.
- OpenAI-compatible: `export OPENAI_API_KEY=...` then `qcode --provider openai --model gpt-5`. Thinking uses reasoning effort where the model offers it.
- OpenCode Go: `export QCODE_API_KEY=...` then `qcode --provider opencode-go --model kimi-k3`. Options vary per model.
- Flags can also come from QCODE_PROVIDER, QCODE_MODEL, QCODE_THINKING, QCODE_BASE_URL, QCODE_API_KEY.

```
qcode --provider ollama --model qwen3:8b --thinking high "reply with OK"
qcode --provider opencode-go --model deepseek-v4-flash --thinking high "reply with OK"
```

Tip

On `main`, /model and /maxsteps remember your choice. Other tabs, demo, one-shot, and piped runs keep changes for that session only.

One-shot and demo

```
qcode "explain this repository"
qcode --json-events "run the tests" 2>events.jsonl
printf '%s\n' 'explain this repository' | qcode
qcode --demo "show me how qcode works"
```

Put flags before one-shot prompts or piped text. Empty stdin fails. Answers use stdout; activity uses stderr. Demo tools are mocked; /export still writes a file.

6. Working with multiple agents

Use extra agents to do independent jobs at the same time, for example one writing code while another reads tests. Each tab keeps its own history, draft, queue, model, and tool choices.



Figure 6: /agent list. Enter switches to the highlighted agent.

Command	What it does for you
/agent	Create an agent (same as /agent new).
/agent new [model]	Create an agent, optionally with a model ID.
/agent list	Pick and switch with Up/Down + Enter.
/agent switch <id>	Jump directly to one agent.
/agent rename <id> <name>	Give an agent a new display name.
/agent cancel <id>	Stop that agent's current work.
/agent close <id>	Cancel work and close the tab after the run exits (--yes skips confirm).

- Switch with Ctrl+PageUp/PageDown (Alt+, and Alt+. also work).
- Ask the main agent to split work and it can coordinate helpers; you can also create tabs yourself with /agent.
- /agent and /agent new open the current provider's model picker. /agent new <model> takes a model ID, then /agent rename <id> <name> sets its display name.
- Queues are per agent (up to 16 waiting each). A slow helper never blocks `main`.
- When main delegates a task, its current run ends after accepting the assignment and the helper continues in its tab. Send a follow-up to main to ask for findings or a consultation. Full recorded findings remain searchable after closing a helper or using /new.
- Main can use close_agent with one agent_id in normal execution mode. Closure returns immediately without ending main's turn. Pending tasks, steering, and interactions are cancelled; a running helper shows closing until its run exits and still counts toward the agent limit. The selected closed tab returns to main. Closing helpers are excluded from saved resumable tabs; recorded work remains searchable and exportable.

7. Planning before coding

Plan mode investigates without changing files. Enter it while idle, let qcode study the code, answer any design questions, review the submitted plan, then build it.



Figure 7: Plan mode. The prompt shows (Plan) and the status bar shows [MODE PLAN] until you act or leave.

1. Type `/plan` to enter. The status bar shows PLAN.
2. Describe the task. Answer focused questions if qcode asks (you can type a custom answer).
3. Type `/plan show` to re-read the latest plan any time.
4. Type `/plan act` to build the latest plan, or `/plan off` to leave without building.

Tip

`/new` clears the saved plan. Plan commands need an idle agent with no running or queued work.

8. Skills and SkillPlan

Using skills

Skills are reusable instruction bundles, for example a team review checklist. Turn them on per tab with `/skill`. Only selected skills are shared with the assistant. You can also run `/skill name1,name2` directly, or `/skill none` to clear them.

- Type `/skill` to see names plus one-line summaries. Type to filter, Up/Down or PgUp/PgDn to move, Space to toggle, Enter to apply, Esc to leave, Ctrl+C to cancel.
- Your account, workspace, and configured folders can each provide skills.
- New files appear in `/skill` without restarting.
- The source repository's docs/skills collection is optional. Add its absolute directory to [skills] paths in config.toml, or copy a chosen folder into a built-in root; release binaries do not bundle these files.
- Use [skills] autoload_paths to select every valid skill in a directory for new runs, including one-shot and piped prompts. Resumed tabs retain their saved selection.

Creating a skill safely

Use `/skillplan` alone or with a rough idea to turn it into a reviewed skill. It asks when the skill applies, its workflow, inputs and outputs, constraints, validation, name, and scope, with follow-ups when an answer leaves a decision open.

```
/skillplan review database migrations before deployment
```

- `/skillplan show` reviews the draft. `/skillplan create` writes it. `/skillplan off` leaves without writing.
- Names allow 1-64 lowercase letters, digits, hyphens, or underscores; files must fit 64 KiB. qcode never overwrites an existing SKILL.md.
- After creation, run `/skill` to enable the new skill.
- SkillPlan can read context but cannot run shell commands or change files until you approve creation.

9. Remembering preferences with /learn

Use `/learn` after a good answer to save a durable preference or reusable procedure. qcode shows a preview and asks `Apply these global learning changes? [y/N]`. Nothing is saved unless you answer `y` or `yes`; Enter, `n`, or Ctrl+C cancels.

Command	What it does for you
/learn	Propose memory from recent conversation.
/learn list	Show saved items and IDs.
/learn forget <id>	Review then delete one item.
/learn compact	Merge duplicates (asks first, backs up).

- Memory is per user on this device and shared across folders.
- Keep entries reusable: state the language or condition when they apply.
- Review proposals: secrets are rejected, but you decide what is worth keeping.

10. Tools, web fetch, and search

Choosing tools

Use /tool to filter and toggle the active tab's file, shell, image, skill, and web tools. Space toggles; Enter applies. Or use /tool <name> on|off, for example /tool shell off. New conversations restore tool defaults. Plan and Skill Plan show only their allowed tools; /interactive controls questions.

Using web access

Web tools read public pages and search results as text. Both start enabled outside sandbox mode and disabled when a sandbox is active, in interactive and one-shot runs. Use `/tool` to change the active tab's selections. The web_search backend setting does not change tool enablement.

- Type `/tool` to toggle `web_fetch` and `web_search` independently, or use `/tool web_fetch off` to disable fetching.
- `/new` and fresh launches restore these defaults. Resume restores saved selections, and helpers created by main inherit its tools; an enabled web tool allows sandbox networking.
- Use them for docs and public references. They do not run page scripts and do not open a browser.

Tip

If a page needs login or runs an app, fetch will not see it. Paste the relevant text into qcode instead.

11. Sessions: autosave, resume, and export

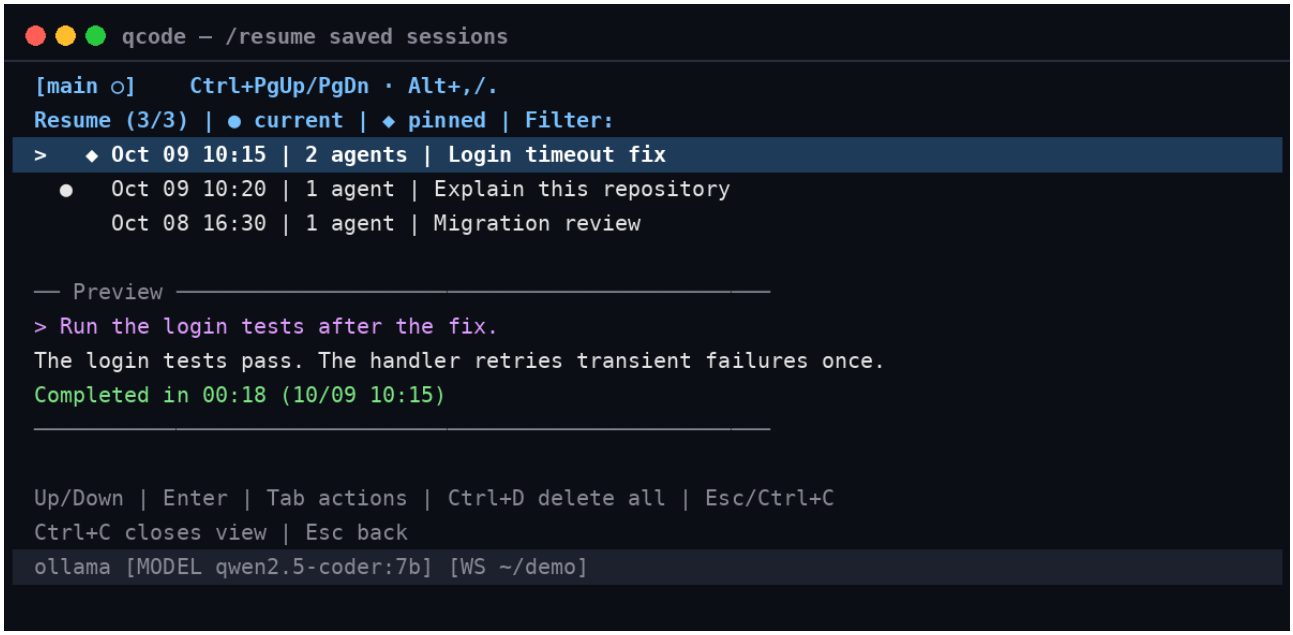


Figure 8: /resume previews the saved active tab. Pinned sessions sort first; Tab opens Rename, Pin/Unpin, and Delete, while Ctrl+D opens Delete all.

- Sessions autosave about every two seconds, after agent work and commands, and on clean exit or session switch. Each launch starts its own session.
- Type /resume to reopen a session for this folder. A separate marker identifies the current session; selecting it keeps it open. Entries show a preview and activity time. Pinned sessions appear first; prompt activity orders each group. Tab opens Rename, Pin/Unpin, and Delete; Ctrl+D opens Delete all from the list. Up/Down chooses an action; Enter applies it. Finish or cancel running work before switching or deleting the current session. Concurrent processes can restore the same session, with the last save winning.
- Ctrl+D opens Delete all with Cancel selected by default. Cancel or Esc returns to the list with its filter and selection intact. It deletes every saved session in this workspace, including sessions hidden by the filter, along with names and pins. Other workspaces are unaffected. If the current session is saved, it starts fresh after the other removals succeed; an unsaved current conversation stays open. Failures refresh the list and retain the current conversation.
- Delete requires confirmation with Cancel selected by default. Enter applies the choice; Cancel or Esc returns to actions; Ctrl+C closes the picker. Deleting another session keeps the filter and selects the next match, or the previous match if it was last. Deleting the current session clears all tabs, drafts, history, plans, usage, work history, and added directory grants, then starts fresh with the main agent's current provider, model, thinking, runtime settings, skills, and tool selections. Deletion removes the saved snapshot, custom name, and pin; another process's later autosave may recreate it.
- Resume restores tabs, models, the pinned latest prompt, conversation and styled output, diffs, drafts, reading positions, tool and skill settings, and token totals. Provider credentials come from current configuration.

- Empty and demo/one-shot runs are not saved. `/new` resets only the current tab inside its session.
- After a crash, start qcode in the same folder and use /resume. The last good checkpoint marks unfinished work interrupted; pending tasks and steering are never resubmitted automatically. Inspect current files before continuing, since completed tool changes are not rolled back.
- Sessions live outside your project: ~/.local/state/qcode/sessions on Linux (or $XDG_STATE_HOME), ~/Library/Application Support on macOS, %AppData% on Windows. They keep private file permissions.

Local privacy and redaction

Local text filtering is on by default. Detected credentials, private keys, payment-card numbers, and labelled card security codes or expiry dates appear as [REDACTED] in output, thinking, tool previews, saved sessions, exports, and browser views. Live editable input stays visible while you compose it; submitted history and saved draft copies are filtered.

When you resume an older session, qcode filters and rewrites it before showing the restored conversation. If that rewrite fails, resume stops. Redacted saved text cannot be recovered, so the resumed conversation may need missing context supplied again.

Filtering applies to local text, not requests sent to a model provider, shell commands, workspace file writes, or image contents. Detection can miss unfamiliar formats. Use [redaction] settings for custom patterns, fields, and paths; see docs/privacy.md and docs/configuration.md for coverage and options.

12. Controlling qcode from a browser

Type /remote to control the same session from a phone or browser. Choose Pure Web for a trusted LAN or Tailscale for your tailnet. No auth gives anyone with the URL access; reserve it for open, short-lived demos.

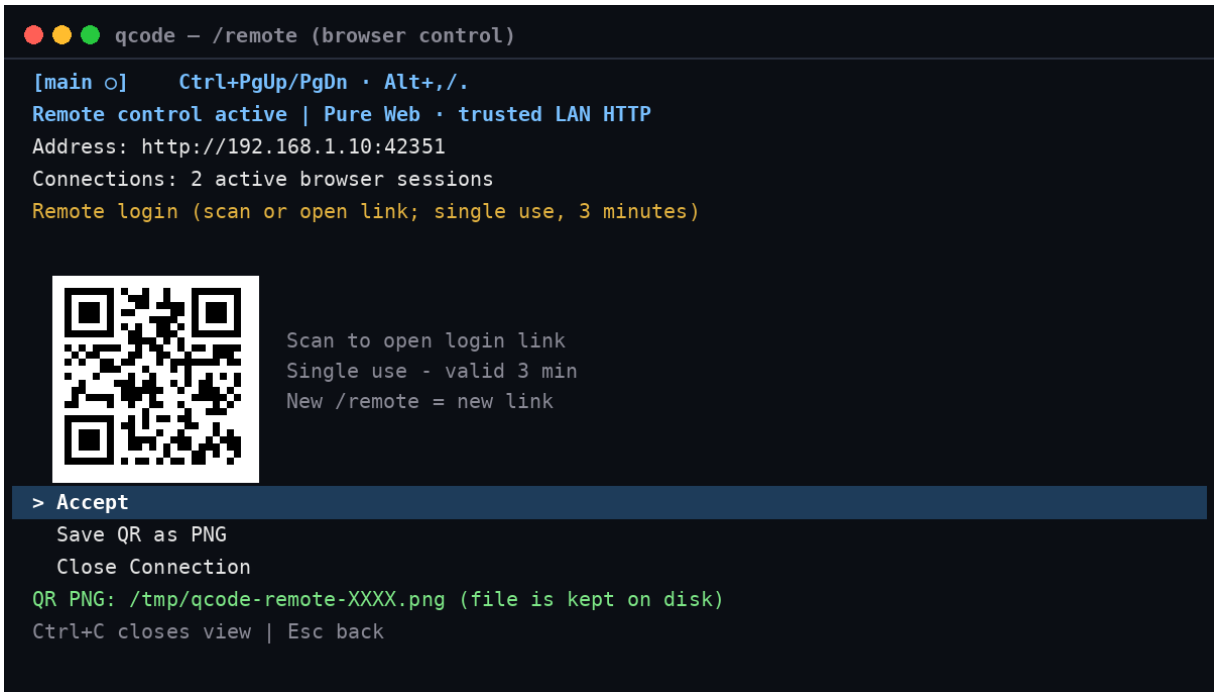


Figure 9: /remote screen. Scan or open the link, then Accept. Save QR as PNG if it does not fit.

1. Type `/remote` and choose how to connect. For multiple networks, pick the interface to share.
2. Scan or open the link on the other device. Links are single-use and last 3 minutes. A new link replaces an unused one; existing browsers stay connected.
3. If the code does not fit, choose `Save QR as PNG` and open the shown file path (in WSL, convert it with `wslpath -w <path>`). The file is kept on disk.
4. Choose Accept to keep the connection open. Run `/remote` again for one more device; existing browsers stay connected.
5. To stop all browsers, choose Close Connection and confirm, or quit qcode. Used and expired links tell you to run `/remote` again.

- Browser Enter and Steer update the busy task; Queue submits a separate task. Tab navigates controls normally.
- Remove cancels pending input; Cancel active work stops the running task. Rejected submissions keep your draft.
- The selected agent's latest received prompt stays below the browser header, up to three lines, while the transcript scrolls. It updates only when a task starts or steering is delivered.
- Beginning and Latest navigate transcript boundaries. Home/End do the same outside text fields, command panels, and the expanded pending list; text fields retain their normal editing keys.
- Reloading the same browser tab resumes access; a fresh browser needs a fresh link. /export downloads the same HTML as in the terminal.
- Use only on networks you trust. Pure Web is plain HTTP on your LAN.

13. Settings you actually change

Most daily choices live in the terminal (`/model`, `/maxsteps`, `/statusline`, `/theme`, `/tool`). Use a config file only for defaults you always want.

Terminal colors and workspace visibility

Use /theme to preview terminal palettes with Up/Down. Enter applies and saves the choice; Esc or Ctrl+C cancels. Default (Auto) follows your terminal. The 22 dark and light choices include Catppuccin, Dracula/Alucard, Gruvbox, Solarized, Nord, Tokyo Night, One, Rosé Pine, Everforest, Kanagawa, and Ayu. The setting is terminal-wide and saves from any agent tab. The browser's pinned prompt follows the theme's accent color and background.

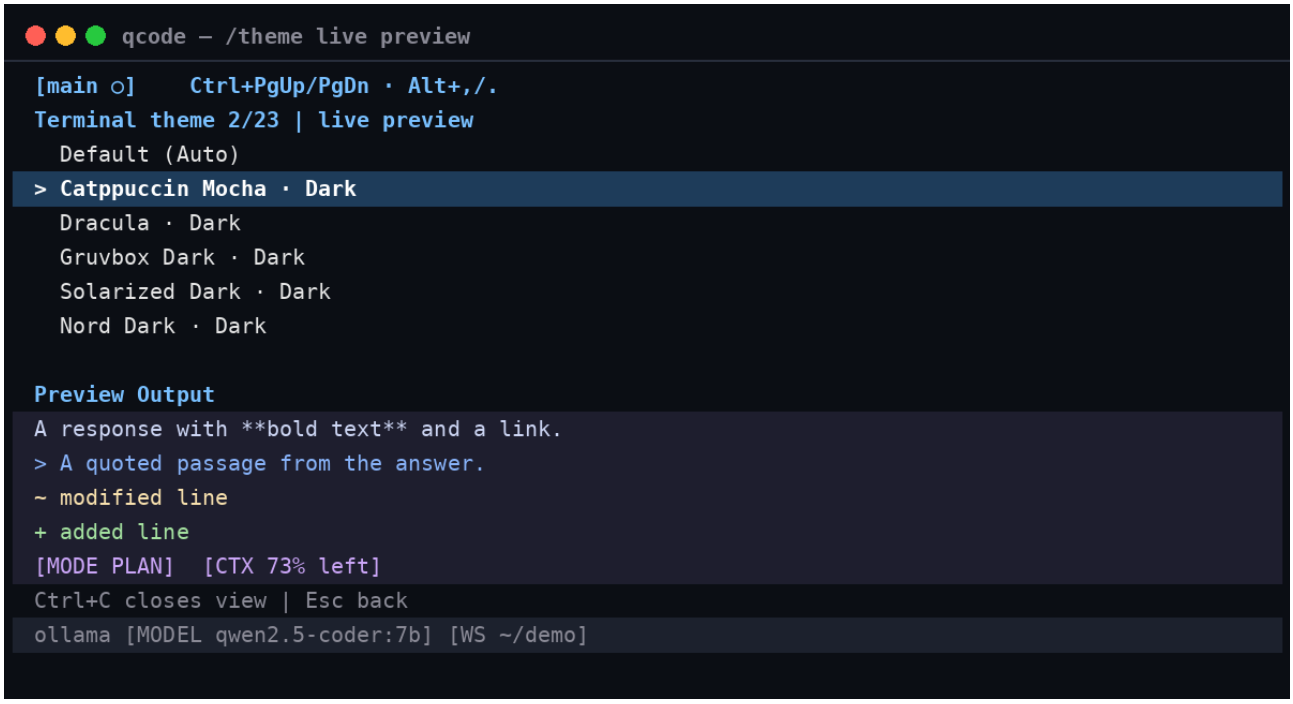


Figure 10: /theme previews colors for responses, diffs, and status. Enter applies and saves; Esc or Ctrl+C leaves the current choice.

WS uses available status bar space and keeps the final two complete folders when possible, using a second row as needed. Home-relative paths retain ~/; omitted parents appear as an ellipsis folder, for example ~/.../work/foo. If the final folder name must be clipped, its ending gets an ellipsis: ~/workspace/this-folder-is... (the final dots shorten the name).

TOK shows provider-reported session input/output counts. The combined total uses a sigma marker, or T: in ASCII mode, whenever it fits one or two status rows. If usage is missing, unknown or a question mark indicates incomplete counts. /new resets the active agent's totals; changing models keeps them.

Open /statusline, move with Up/Down, toggle with Space, and apply with Enter. Or type /statusline tok off, /statusline show, or /statusline reset. Changes on main save statusline_hidden in your user config while preserving comments and unrelated tables, including redaction and skills settings.

```
provider = "ollama"
model = "qwen2.5-coder:7b"
max_steps = 32
theme = "catppuccin-mocha"
sandbox = true
```

- Copy `config.toml.example` to your user config location; missing files are ignored. Priority, highest first: file beside the program, user config (~/.local/etc/qcode on Linux, ~/Library/Application Support on macOS, %AppData% on Windows), system config (/usr/local/etc/qcode), then etc/ beside the program.
- Command flags beat environment variables. Both beat config files. Config files beat built-ins. `--api-key` beats config and environment keys.
- Prefer environment variables for keys: QCODE_API_KEY or OPENAI_API_KEY. Keep key files private (mode 0600).
- Common knobs: thinking level, agent_timeout like "5m", interactive true/false, statusline_hidden, web_search backend, learning budget, skills paths.
- Restart after changing the web backend. /model, /maxsteps, and /statusline on main save automatically; worker-tab changes to these settings stay within the session.

Sandbox and trusted commands

On Linux, use --sandbox with bubblewrap installed and check the startup notice to confirm isolation is active. Tools can change only approved directories; home is hidden. Fresh sandbox sessions start with both web tools off and networking blocked. Enabling either through /tool allows networking for native web tools and sandboxed shell commands. Disabling both or using /new blocks it again. If sandbox setup fails, the terminal offers to continue without it or leave; one-shot runs warn and continue without it. Continuing without a sandbox starts with both web tools enabled.

```
qcode --sandbox --sandbox-command-path ~/.local/bin

# Persistent defaults in config.toml:
sandbox = true
sandbox_command_paths = ["~/.local/bin", "~/go/bin"]
```

Configured sandbox = true and --sandbox use the same defaults; --sandbox=false overrides configuration. Trusted command directories are mounted read-only and kept across /new. Missing, invalid, duplicate, root, or protected directories produce warnings. Additional writable directories require your approval and are cleared by /new. Temporary shell files do not survive separate sandbox calls; use workspace files for results you need later.

Context capacity

CTX shows remaining capacity, with ~ for an estimate or unknown when no limit is available. Use --context-window 32768 or context_window = 32768 for an unlisted model; this sets terminal-agent accounting and compaction capacity without changing the provider's allocation. Switching models clears the override. --disable-auto-compact disables automatic summaries in terminal sessions; /compact remains available. One-shot runs use provider capacity and the built-in 80% threshold.

14. Troubleshooting and tips

Command	What it does for you
No colors?	Unset NO_COLOR to enable colors. Set NO_COLOR=1 to disable them; QCODE_ASCII=1 uses ASCII glyphs while retaining color.
Narrow window?	The path shortens first; the bar wraps to a second left-aligned line. Low-priority parts drop only if two lines still overflow. Hide more with /statusline.
Slow session?	Try /compact, then /new for a fresh tab context.
Need detail?	Try /verbose. For step limits use /maxsteps N.
Lost output?	Scroll with PageUp/PageDown. /history finds old answers. /export saves HTML.
Resume failed?	Read the inline error; finish or cancel local running agents before switching. Fix snapshot, redaction-rewrite, or save errors, then retry. Other processes do not lock the session.
Waiting a long time?	Read the model operation and elapsed timer above the status bar. Use Ctrl+C from the composer to cancel; tool activity appears in the transcript.
Text is redacted?	Detected sensitive text is replaced locally by [REDACTED]. Saved redactions are permanent; review docs/privacy.md for scope.
Remote blocked?	On Linux run once: sudo tailscale set --operator=$USER. Then retry /remote.

Tip

Good defaults for most users: local model for drafts, stronger hosted model for final edits, Plan mode for risky changes, one agent per independent job.

15. Cheat sheet (copy this page)

Command	What it does for you
/help	Help for all or one command.
/bash <cmd>	Run a shell command.
/model	Switch model by name only (+ thinking).
/agent ...	Create, list, switch, rename, cancel, close.
/plan ...	Plan, show, act, off.
/skill	Enable instruction bundles.
/skillplan ...	Draft, show, create, or leave skill design.
/interactive	Let normal work ask up to 3 questions per prompt.
/learn ...	Save/list/forget/compact preferences.
/tool	Choose file, shell, image, skill, and web tools.
/resume, /new	Reopen session / reset tab.
/history	Find and re-read answers.
/diff	Expand saved diffs.
/compact	Summarize old context.
/verbose	Show/hide detailed traces.
/export	Save pretty or raw HTML.
/remote	Browser control via QR link.
/statusline	Choose status bar parts.
/theme	Preview and save terminal colors.
/maxsteps	Show/set step limit.
/clear	Clear view and redraw header.
/quit, /exit	Save session and exit.

One-shot recipes:

```
qcode --model qwen2.5-coder:7b "explain this repository"
qcode --provider openai --model gpt-5 "summarize the diff"
qcode --demo "show me how qcode works"
```

About this manual

This manual teaches use, not construction. It omits source layout, network internals, and developer extension points on purpose. For command details, type `/help` inside qcode. Generated October 14, 2026 as version dev. Screenshots are illustrative examples rendered for print clarity.

Chapters in this manual

- How to use this manual
- 1. Getting started
- 2. Touring the terminal
- 3. Asking, queueing, and history
- Use cases: understand and fix; plan and divide work; reuse and continue
- 4. Slash commands at a glance
- 5. Choosing models and providers
- 6. Working with multiple agents
- 7. Planning before coding
- 8. Skills and SkillPlan
- 9. Remembering preferences with /learn
- 10. Tools, web fetch, and search
- 11. Sessions: autosave, resume, and export
- 12. Controlling qcode from a browser
- 13. Settings you actually change
- 14. Troubleshooting and tips
- 15. Cheat sheet